Kolej krzesełkowa – mini sprawozdanie (v3.9.1)

Środowisko

- Linux / WSL2 (Ubuntu) lub uruchomienie przez SSH
- Kompilator: GCC
- Mechanizmy IPC: System V (Shared Memory, Semafory, Kolejki komunikatów)

---

1. Opis zadania (wymagania)

W górskiej miejscowości działa krzesełkowa kolej linowa czynna w okresie letnim, obsługująca m.in. rowerowe trasy zjazdowe.

- Z kolei korzystają **piesi** oraz **rowerzyści**.
- Kolej w modelu składa się z **18 „rzędów" po 4 krzesełka** (łącznie **72 krzesełka / miejsca**). Jednocześnie w górę jedzie **9 rzędów**, więc może być zajętych maksymalnie **36 krzesełek** (druga połowa wraca pusta w dół).
- W jednym rzędzie (4 krzesełka) mogą jechać maksymalnie:
  - **2 rowerzystów** (2+2 miejsca), albo
  - **1 rowerzysta + 2 pieszych** (2+1+1), albo
  - **4 pieszych**.
- Turyści/rowerzyści przychodzą w losowych momentach (nie wszyscy muszą korzystać z kolei).
- Wejście na teren odbywa się po zakupie biletu/karnetu w kasie.
- Karnety: jednorazowe, czasowe (Tk1/Tk2/Tk3) lub dzienne.
- Zniżka 25%: dzieci < 10 lat i seniorzy 65+.
- Dzieci < 8 lat są pod stałą opieką osoby dorosłej; dorosły może opiekować się max. **dwójką** dzieci (4–8 lat).
- Bramki:
  - wejście na teren dolnej stacji: **4 bramki** (kontrola karnetu),
  - wejście na peron: **3 bramki** otwierane przez **pracownik1**.
- Na terenie dolnej stacji (między 1. bramkami a peronem) może przebywać maksymalnie **N osób**.
- Stacja dolna: **pracownik1**, stacja górna: **pracownik2**.
- Awaria: pracownik1 lub pracownik2 zatrzymuje kolej (**sygnał1**), a wznowienie (**sygnał2**) następuje po komunikacji z drugim pracownikiem i potwierdzeniu gotowości.
- Po osiągnięciu czasu zamknięcia **Tk**: bramki przestają akceptować karnety, osoby już na peronie muszą zostać przetransportowane na górę, a następnie po **3 sekundach** kolej ma się wyłączyć.
- VIP (~1%): wchodzą na teren dolnej stacji **bez kolejki**, używając karnetu.

---

2. Uruchomienie

2.1. Kompilacja

```
make
```

2.2. Start symulacji

Program uruchamia się poleceniem:

```
./main N CZAS [LIMIT_UTWORZONYCH] [LIMIT_AKTYWNYCH] [KARNETY_MASK]
```

Gdzie: - `N` – limit osób, które mogą jednocześnie przebywać **na terenie dolnej stacji** (między bramkami wejściowymi a peronem). - `CZAS` – czas pracy symulacji w sekundach; po jego upływie uruchamia się procedura końca dnia (CLOSING → DRAINING → SHUTDOWN). - `LIMIT_UTWORZONYCH` *(opcjonalny)* – maksymalna łączna liczba wygenerowanych klientów (0 = bez limitu). - `LIMIT_AKTYWNYCH` *(opcjonalny)* – maksymalna liczba jednocześnie aktywnych klientów (0 = bez limitu). - `KARNETY_MASK` *(opcjonalny)* – dozwolone typy karnetów: liczba (maska bitowa 1–31) lub nazwa (`jednorazowy`, `jednorazowy,tk1`, `wszystkie`).

Przykład (podstawowy):

```
./main 20 30
```

Przykład (z limitami generatora):

```
./main 20 30 500 100
```

2.3. Podgląd logów (WSL2 / SSH)

Najwygodniej w WSL2/SSH uruchomić wersję „demo” w tmux (nie wymaga GUI):

```
./demo_tmux_windows.sh 20 30
```

(Uwaga: przewijanie w tmux: `Ctrl+b` potem `[`; wyjście: `q`.)

---

3. Architektura programu

3.1. Procesy

- **main** – koordynator: tworzy IPC, uruchamia procesy, zarządza fazami dnia i końcem symulacji.
- **generator** – tworzy procesy klientów (fork+exec) i loguje parametry klientów.
- **kasjer** – sprzedaje karnety (odmawia w CLOSING/DRAINING).
- **bramki (4× bramka1)** – kontrola wejścia na teren dolnej stacji (weryfikacja karnetu + limit N).
- **pracownik1** – obsługa stacji dolnej, w tym „bramki peronu” (3 wirtualne bramki).

- **wyciąg** – symulacja obiegu **18 rzędów × 4 krzesełka (72 miejsca)**: w danym momencie **9 rzędów** jedzie w górę (max **36 zajętych krzesełek/miejsc**), a 9 wraca pusta w dół.
- **pracownik2** – obsługa stacji górnej oraz udział w procedurze STOP/START.
- **sprzątacz** – proces ochronny: w razie awarii/crasha czyści IPC i dobija procesy.
- **klient** – cykl życia klienta: kasa → bramki wejściowe → peron → wyciąg → zjazd → (powtórka lub zakończenie).

3.2. Przepływ klienta (skrót)

1. **Decyzja „czy korzysta"** – część klientów odchodzi bez kupowania (zgodnie z wymaganiem „nie wszyscy muszą korzystać").
2. **Kasa** → zakup karnetu.
3. **Bramka wejściowa (1. strefa)** → wejście na teren dolnej stacji (limit N).
4. **Peron (pracownik1)** → kontrola wejścia na peron przez 3 bramki.
5. **Wyciąg** → żądanie przejazdu, wejście na krzesełko (do 4 miejsc/slotów), dojazd na górę.
6. **Góra + zjazd** → klient „zjeżdża" losową trasą i wraca na dół.
7. Koniec: gdy karnet nieważny / koniec dnia / odmowa w którejś strefie.

3.3. IPC w skrócie

- **Pamięć dzielona (SHM):** globalny stan symulacji (faza dnia, liczniki osób, statystyki, PIDy procesów, flagi awarii/panic).
- **Semafory:**
  - `SEM_TEREN` – limit **N** osób na dolnej stacji.
  - `SEM_PERON` – limit miejsc na pojedynczym krzesełku (4 „sloty").
  - `SEM_MUTEX_SHM` – mutex do krytycznych aktualizacji SHM.
  - `SEM_BARIERA_AWARIA` – bariera blokująca procesy podczas awarii.
- **Kolejki komunikatów (MQ):**
  - Kasa: żądania i odpowiedzi.
  - Bramki: żądania i odpowiedzi.
  - Peron: żądania klientów + odpowiedzi pracownika1.
  - Wyciąg: żądania BOARD + odpowiedzi BOARD/ARRIVE.
  - Pracownicy: komunikacja STOP/START.

3.4. VIP

VIP jest realizowany przez **wydzieloną bramkę wejściową (nr 1)**, z której korzystają wyłącznie klienci VIP. Klienci „normalni" wybierają bramki 2–4. Dzięki temu VIP nie czeka w tej samej kolejce.

3.5. Awaria (STOP/START)

- **Sygnał1 = SIGUSR1 (STOP):** inicjator (pracownik1 lub pracownik2) zatrzymuje kolej (ustawia flagę awarii i blokuje ruch).

- **Sygnał2 = SIGUSR2 (START):** wznawia **ten sam pracownik, który zatrzymał** – wysyła komunikat do drugiego pracownika i czeka na odpowiedź gotowości; potem odblokowuje barierę i uruchamia kolej.

3.6. Koniec dnia (CLOSING → DRAINING → SHUTDOWN)

- **CLOSING:** nowe osoby nie są wpuszczane (bramki/kasa odmawiają), karnety „umierają" od `czas_konca_dnia`.
- **DRAINING:** wyciąg przewozi wszystkich, którzy **weszli już na teren dolnej stacji (między BRAMKA1 a peronem)** oraz tych, którzy są już **na peronie / na krzesełkach**; po opróżnieniu terenu/peronu/krzesełek czeka **3 sekundy** i kończy.
- **SHUTDOWN:** main zamyka pozostałe procesy i czyści IPC.

---

4. Logi

Logi są kierowane do plików w katalogu `output/` (podział pod wygodny pokaz w 4 panelach):

- `output/main.log` – start, fazy dnia, shutdown, statystyki
- `output/pracownicy.log` – STOP/START + handshake pracowników
- `output/generator.log` – tworzenie klientów i ich parametry
- `output/kasa.log` – sprzedaż karnetów
- `output/bramki.log` – decyzje bramek (OK/ODRZUT)
- `output/wyciag.log` – start pracy wyciągu, awarie, drenowanie
- `output/klienci.log` – „ścieżka życia" klientów (kasa/bramki/peron/BOARD/ARRIVE/koniec)
- `output/sprzatacz.log` – awaryjne sprzątanie (jeśli potrzebne)

---

5. Testy (skrypty)

Poniżej znajduje się seria **9 testów** (skrypty w katalogu `tests/`), których celem jest sprawdzenie działania symulacji w warunkach "brzegowych" oraz potwierdzenie, że nie dochodzi do: - **blokady / zakleszczenia** (np. przez semafory, MQ, DRAINING), - **przekroczeń limitów** (np. limit miejsc/slotów, limit osób na terenie `N`), - niekontrolowanego "rozjazdu" logiki (np. BOARD bez ARRIVE), - problemów końca dnia (CLOSING → DRAINING → SHUTDOWN).

Każdy test zapisuje wyniki w `tests/results/<nazwa_testu>_<timestamp>/summary.txt`. Do raportu dołączone są zrzuty ekranu z plików `summary.txt` (oraz ewentualnie z logów `output/*.log`).

Test 1 – Limity rzędów/krzesełek i zasady sadzania (18×4=72, 9×4=36)

Cel: potwierdzenie modelu 72 krzesełek (w obiegu) i limitu 4 slotów na krzesełko (pieszy=1, rowerzysta=2; dzieci liczone jako osoby w grupie), oraz spójności przejazdów (**BOARD == ARRIVE**).

Uruchomienie (przykład):

`./tests/test1_krzeselka_i_sloty.sh 120 10`

Warunki wejściowe: `N=120`, `CZAS=10` (intensywny ruch – dużo zdarzeń w krótkim czasie).

Sprawdzenia (co jest w `summary.txt`): 1. Linia startowa wyciągu: `RZEDOW=18` oraz `KRZESLA_W_RZEDZIE=4` → potwierdza parametry modelu. 2. "Twarde" liczenie zdarzeń z `klienci.log`: - liczba wystąpień `KLIENT <id>: BOARD` i `KLIENT <id>: ARRIVE`, - wymagane: **BOARD == ARRIVE** (brak "znikających" przejazdów). 3. Maksymalny rozmiar grupy w logach klientów (`max_rozmiar_grupy`) – wymagane: **≤ 4**. 4. Zgodność z raportem dziennym: `Liczba przejazdów` == BOARD == ARRIVE.

Oczekiwany wynik: `OK: parametry 72/4`, `OK: BOARD == ARRIVE`, `OK: max_rozmiar_grupy <= 4`, oraz zgodność z raportem.

Test1 – potwierdzenie limitów

```
TEST1(HARD): krzesełka i sloty
N=120, CZAS=10

[WYCIAG] Start:
[22:25:58][PID 378164] WYCIAG: Start (INTERWAL=1ms, PRZEJAZD=9ms, RZEDOW=18, SLOTY/RZAD=4)
Parsed: RZEDOW=18, SLOTY/RZAD=4
OK: parametry wyciągu zgodne (18 rzędów / 4 krzesełka na rząd).

[STAT - HARD] Zdarzenia z klienci.log (wystąpienia wzorca):
BOARD events:  2164
ARRIVE events: 2164
OK: BOARD == ARRIVE (spójne przejazdy).

[CHECK] Max rozmiar_grupy (sloty) z klienci.log:
max_rozmiar_grupy=4
OK: max_rozmiar_grupy <= 4.

[RAPORT] Liczba przejazdów z raportu: 2164
OK: raport == BOARD == ARRIVE.

[SAMPLE] BOARD (5 przykładów):
[22:25:58][PID 378170] KLIENT 1: BOARD - wsiadam na krzesełko (sloty=1)
[22:25:58][PID 378171] KLIENT 2: BOARD - wsiadam na krzesełko (sloty=2)
[22:25:58][PID 378173] KLIENT 4: BOARD - wsiadam na krzesełko (sloty=1)
[22:25:58][PID 378175] KLIENT 5: BOARD - wsiadam na krzesełko (sloty=1)
[22:25:58][PID 378176] KLIENT 6: BOARD - wsiadam na krzesełko (sloty=2)

[SAMPLE] ARRIVE (5 przykładów):
[22:25:58][PID 378170] KLIENT 1: ARRIVE - jestem na górze (przejazd=1)
[22:25:58][PID 378171] KLIENT 2: ARRIVE - jestem na górze (przejazd=1)
[22:25:58][PID 378173] KLIENT 4: ARRIVE - jestem na górze (przejazd=1)
[22:25:58][PID 378172] KLIENT 3: ARRIVE - jestem na górze (przejazd=1)
[22:25:58][PID 378175] KLIENT 5: ARRIVE - jestem na górze (przejazd=1)
```

Test 2 – Limit osób na terenie (N) + VIP + "nie korzysta"

Cel: sprawdzenie trzech wymagań funkcjonalnych: 1) limitu liczby osób na terenie dolnej stacji (`N`) — bramka używa blokującego `sem_wait`, więc klienci czekają (nie są odrzucani), 2) obsługi VIP (dedykowana bramka VIP-only), 3) poprawnego zachowania klientów, którzy "przychodzą i odchodzą" (nie korzystają z kolei).

Uruchomienie (przykład):

```
./tests/test2_limitN_i_vip.sh 6 10
```

Warunki wejściowe: małe `N`, krótki czas (intensywny napływ klientów).

2A) Limit N – blokowanie na semaforze terenu

Bramka realizuje limit `N` przez blokujący `sem_wait` na semaforze terenu — gdy teren jest pełny, klienci czekają w kolejce do bramki zamiast być odrzucani.

Sprawdzenie: test liczy `bramki_ok_count` (ile klientów przeszło przez bramki). Przy małym `N` i dużym ruchu klienci muszą czekać, ale symulacja nie blokuje się na stałe.

Oczekiwany wynik (część limit N): - bramki przepuszczają klientów (`bramki_ok_count > 0`), - symulacja nadal działa (brak deadlocka na semaforach i MQ).

2B) VIP – brak kolejki przez dedykowaną bramkę

Sprawdzenie: w `summary.txt` jest: - `vip_ok_count > 0` oraz przykład wpisu z `BRAMKA1: OK ... vip=1`.

Oczekiwany wynik: VIP przechodzi przez dedykowaną bramkę, bez "utknięcia" w kolejce zwykłych bramek.

2C) "Nie korzysta"

Sprawdzenie: zliczane są linie w `klienci.log` typu: - `odchodzi - dziś nie korzysta z kolei ...`

Oczekiwany wynik: `count > 0` (dla `PROC_NIE_KORZYSTA=10%` zwykle pojawi się kilka/kilkadziesiąt wpisów, zależnie od liczby klientów w czasie testu).

Test 2 – Limit osób na terenie

```
TEST2: limit N i VIP
N=6, CZAS=15

[LIMIT N] Bramka używa blokującego sem_wait (N=6) - klienci czekają, nie są odrzucani.
bramki_ok_count=2328

[VIP] Dedykowana BRAMKA1 (VIP-only):
vip_ok_count=26
Przykład:
[22:26:26][PID 399462] BRAMKA1: OK - pid=399497 karnet=26 grupa=2 vip=1

[NIE KORZYSTA] Klienci, którzy rezygnują (case-insensitive):
count=2610
[22:26:26][PID 399479] KLIENT 10: odchodzi - dziś nie korzysta z kolei (los=1 < 10%)
[22:26:26][PID 399477] KLIENT 8: odchodzi - dziś nie korzysta z kolei (los=7 < 10%)
[22:26:26][PID 399477] DZIECKO 1: razem z opiekunem -> ODCHODZI
[22:26:26][PID 399477] DZIECKO 2: razem z opiekunem -> ODCHODZI
[22:26:26][PID 399504] KLIENT 32: odchodzi - dziś nie korzysta z kolei (los=3 < 10%)
[22:26:26][PID 399521] KLIENT 45: odchodzi - dziś nie korzysta z kolei (los=8 < 10%)
[22:26:26][PID 399525] KLIENT 49: odchodzi - dziś nie korzysta z kolei (los=0 < 10%)
[22:26:26][PID 399524] KLIENT 48: odchodzi - dziś nie korzysta z kolei (los=1 < 10%)
[22:26:26][PID 399538] KLIENT 61: odchodzi - dziś nie korzysta z kolei (los=3 < 10%)
[22:26:26][PID 399546] KLIENT 69: odchodzi - dziś nie korzysta z kolei (los=4 < 10%)
```

Test 3 – Awaria STOP/START (pracownik1/pracownik2) + brak zakleszczeń

Cel: potwierdzenie wymagań awarii: - pracownik1 lub pracownik2 zatrzymuje kolej (STOP – sygnał 1), - inicjator komunikuje się z drugim pracownikiem, - po potwierdzeniu GOTOWY kolej jest wznawiana (START – sygnał 2), - brak deadlocka (wyciąg "czeka na wznowienie" i wraca do pracy).

Uruchomienie (przykład):

```
./tests/test3_awaria_stop_start.sh 120 35
```

Procedura testu (co robi skrypt): 1. Uruchamia symulację. 2. Wysyła SIGUSR1 (STOP/AWARIA) do `main` — main przekazuje do pracowników. 3. Po chwili wysyła SIGUSR2 (START/WZNOWIENIE) do `main`. 4. Czeka na zakończenie.

Sprawdzenia (w `summary.txt`): - w `main.log` muszą pojawić się wpisy: `SYGNAŁ SIGUSR1 ... AWARIA` i `SYGNAŁ SIGUSR2 ... WZNOWIENIE`. - w `pracownicy.log` musi być sekwencja: `STOP (inicjator)` → `GOTOWY (STOP)` → procedura wznowienia.

Oczekiwany wynik: poprawny handshake STOP/START między pracownikami, main odbiera oba sygnały.

Test 3 – Awaria STOP/START

```
TEST3: awaria STOP/START
N=120, CZAS=35

[MAIN] Sygnały odebrane:
[22:26:54][PID 425012] SYGNAŁ SIGUSR1 - AWARIA (STOP) -> przekazuję do pracownika
[22:26:56][PID 425012] SYGNAŁ SIGUSR2 - WZNOWIENIE (START) -> przekazuję do inicjatora

[PRACOWNICY] Procedura STOP/GOTOWY/START:
[22:26:54][PID 425016] PRACOWNIK1: STOP (inicjator) - kolej zatrzymana
[22:26:54][PID 425017] PRACOWNIK2: Otrzymano STOP (od P1) - potwierdzam GOTOWY
[22:26:54][PID 425016] PRACOWNIK1: Drugi pracownik GOTOWY (STOP)
[22:26:56][PID 425016] PRACOWNIK1: START (inicjator) - proszę P2 o gotowość
[22:26:56][PID 425017] PRACOWNIK2: Otrzymano START (od P1) - potwierdzam GOTOWY
[22:26:56][PID 425016] PRACOWNIK1: P2 GOTOWY (START)

## Asercje
- main: AWARIA(SIGUSR1)=1 WZNOWIENIE(SIGUSR2)=1
- pracownicy: STOP_initiated=1 GOTOWY=3
```

Test 4 – Koniec dnia: CLOSING → DRAINING → SHUTDOWN (+ 3 sekundy)

Cel: potwierdzenie wymagań końca dnia: - po osiągnięciu Tk bramki nie wpuszczają nowych klientów (CLOSING), - wyciąg przewozi wszystkich, którzy już są "w systemie" (DRAINING), - po opróżnieniu następuje oczekiwanie 3 sekundy i zakończenie (SHUTDOWN).

Uruchomienie (przykład):

`./tests/test4_koniec_dnia_drenowanie.sh 180 40`

Sprawdzenia (w `summary.txt`): - w `main.log` widać przejście przez fazy: `FAZA 1: CLOSING`, `FAZA 2: DRAINING`, `FAZA 3: SHUTDOWN`. - w `wyciag.log` widać: `Drenowanie zakończone - wyłączam za 3s`.

Oczekiwany wynik: brak blokady w DRAINING, procesy kończą się kontrolowanie, IPC jest czyszczone.

Test 4 – Koniec dnia

```
TEST4: koniec dnia CLOSING/DRAINING/SHUTDOWN
N=180, CZAS=40

[MAIN] Fazy dnia (fragment):
[22:28:25][PID 450670] FAZA 1: CLOSING - nie wpuszczamy nowych klientów
[22:28:25][PID 450670]   faza_dnia = CLOSING
[22:28:25][PID 450670] FAZA 2: DRAINING - kończymy transport z peronu (czekam na wyciąg)
[22:28:42][PID 450670] FAZA 3: SHUTDOWN - zamykanie procesów

[WYCIAG] Drenowanie (fragment):
[22:28:39][PID 450676] WYCIAG: Drenowanie zakończone (w_krzesle=0, kolejka=0, peron=0, teren=0) - wyłączam za 3s

## Asercje
- FAZA1_CLOSING=1 FAZA2_DRAINING=1 FAZA3_SHUTDOWN=1
- Wyciąg drenowanie+3s=1
```

Test 5 – Komunikacja end-to-end (kasa → bramka → peron → wyciąg)

Cel: pokazać na jednym przykładzie, że komunikaty przechodzą przez wiele procesów: klient kupuje karnet, przechodzi bramki, wchodzi na peron i dostaje od wyciągu `BOARD/ARRIVE`.

Uruchomienie (przykład):

```
./tests/test5_komunikacja_end_to_end.sh
```

Co skrypt zapisuje do `summary.txt`: - wybiera jednego klienta (id+pid) z `klienci.log`, - pokazuje fragmenty `klienci.log`, `kasa.log`, `bramki.log`, `main.log` dla **tego samego pid** (dowód komunikacji przez MQ), - (jeśli jest `./monitor`) dorzuca snapshot w trakcie działania.

Oczekiwany wynik: w `summary.txt` widać spójny łańcuch zdarzeń dla jednego pid.

```
Wybrany klient (z logów): id=16 pid=476305

## Snapshot MONITOR (w trakcie działania)
```
KOLEJ KRZESEŁKOWA - MONITOR

Faza dnia: OPEN   awaria=0   panic=0
Czas do konca dnia: 25 s   (start=1771882122, koniec=1771882147)

Liczniki: teren=20 peron=0 w_krzesle=3 gora=43 aktywni_klienci=669
Statystyki: wygenerowani=663 przejazdy=26 przychod=23088.75 zl
----
Semafory: TEREN=0 PERON=1 BARIERA_AWARIA=0

MQ: kasa=4/16384 kasa_odp=0/16384 bramka=620/16384 bramka_odp=0/16384
MQ: peron=0/16384 peron_odp=4/16384 wyciag_req=0/16384 wyciag_odp=4/16384 prac=0/16384

PIDy: main=476277 gen=476288 kasjer=476280 wyciag=476283
PIDy: P1=476281 P2=476282 bramki: 476284 476285 476286 476287
```

## Dowody w logach (ten sam pid w wielu procesach)

### klient (output/klienci.log)
[22:28:42][PID 476305] KLIENT 16: start pid=476305 wiek=93 typ=PIESZY vip=0 dzieci=0 (0,0) rozmiar_grupy=1
[22:28:42][PID 476305] KLIENT 16: kupił karnet id_karnetu=7 typ=Dzienny czas_waznosci=86400s vip=0
[22:28:42][PID 476305] KLIENT 16: id_karnetu=7 -> BRAMKA1 nr=3 (vip=0, grupa=1)
[22:28:42][PID 476305] KLIENT 16: BRAMKA1 OK (nr=3) - jestem na terenie
[22:28:42][PID 476305] KLIENT 16: czekam na peron (sloty=1, bramka2=1)
[22:28:42][PID 476305] KLIENT 16: prosi PRACOWNIK1 o wejście na peron (bramka2=1 sloty=1)
[22:28:42][PID 476305] KLIENT 16: PRACOWNIK1 pozwolił wejść na peron
[22:28:42][PID 476305] KLIENT 16: NA_PERONIE (sloty=1) - czekam na BOARD
[22:28:42][PID 476305] KLIENT 16: BOARD - wsiadam na krzesełko (sloty=1)
[22:28:42][PID 476305] KLIENT 16: ARRIVE - jestem na górze (przejazd=1)
[22:28:42][PID 476305] KLIENT 16: zjazd trasą T4 (piesza) (czas=4s)
[22:28:46][PID 476305] KLIENT 16: wróciłem na dół po trasie T4 (piesza)
[22:28:46][PID 476305] KLIENT 16: id_karnetu=7 -> BRAMKA1 nr=2 (vip=0, grupa=1)
[22:29:07][PID 476305] KLIENT 16: koniec (przejazdy=1)

### kasa (output/kasa.log)
[22:28:42][PID 476280] KASJER: klient id=16 pid=476305 wiek=93 typ=PIESZY vip=0 dzieci=0 (0,0)
[22:28:42][PID 476280] KASJER: SPRZEDAŻ id_klienta=16 pid=476305 -> karnet=7 typ=Dzienny cena=75.00 zł vip=0
[22:28:42][PID 476280] KASJER: sprzedano karnet id_karnetu=7 typ=Dzienny cena=75.00 zł dla klienta id=16 (pid=476305)

### bramki (output/bramki.log)
[22:28:42][PID 476286] BRAMKA3: OK - pid=476305 karnet=7 grupa=1 vip=0

### main (output/main.log)
[22:28:42][PID 476277] Kasjer uruchomiony (PID=476280)
[22:28:42][PID 476277] Wyciąg uruchomiony (PID=476283)
[22:28:42][PID 476277] Bramka 1 uruchomiona (PID=476284)
[22:28:42][PID 476277] Bramka 2 uruchomiona (PID=476285)
[22:28:42][PID 476277] Bramka 3 uruchomiona (PID=476286)
[22:28:42][PID 476277] Bramka 4 uruchomiona (PID=476287)
[22:28:42][PID 476277] Generator uruchomiony (PID=476288)
[22:28:42][PID 476277] Wszystkie procesy stałe uruchomione
[22:29:10][PID 476277] Wszystkie procesy potomne zakończone

## Interpretacja (do prezentacji)
- Ten sam pid widoczny w logach kasy i bramek oznacza, że komunikaty MQ zostały odebrane i obsłużone.
- Klient loguje przejścia przez etapy (kasa → bramka → peron → BOARD/ARRIVE), co domyka ścieżkę end-to-end.

## Test 6 – Skrajność: SIGSTOP kasjera i backlog na MQ

**Cel:** zasymulować chwilowe zatrzymanie procesu kasjera (`SIGSTOP`) i pokazać, że: (1) kolejka do kasy się zapełnia (msg_qnum rośnie), (2) po `SIGCONT` system wraca do pracy i backlog maleje.

### Uruchomienie (przykład):

```
./tests/test6_skrajnosci_sigstop_kolejki.sh
```
```

Sprawdzenia: - `summary.txt` zawiera 3 snapshoty z `./monitor`: przed STOP, w trakcie STOP, po CONT, - w `kasa.log` widać, że po wznowieniu znowu sprzedaje, - symulacja kończy się normalnie (brak deadlocka).

Oczekiwany wynik: w trakcie STOP kolejki rosną, po CONT wracają do normy.

```
## Snapshot po SIGCONT (system wraca do pracy)
```

```
KOLEJ KRZESEŁKOWA - MONITOR
------------------------------------------------------------
Faza dnia: OPEN   awaria=0   panic=0
Czas do konca dnia: 18 s   (start=1771882150, koniec=1771882185)
------------------------------------------------------------
Liczniki: teren=30  peron=0  w_krzesle=0  gora=4  aktywni_klienci=577
Statystyki: wygenerowani=572  przejazdy=3  przychod=18690.00 zl
------------------------------------------------------------
Semafory: TEREN=0  PERON=0  BARIERA_AWARIA=0
------------------------------------------------------------
MQ: kasa=0/16384  kasa_odp=0/16384  bramka=409/16384  bramka_odp=0/16384
MQ: peron=0/16384  peron_odp=0/16384  wyciag_req=0/16384  wyciag_odp=3/16384  prac=0/16384
------------------------------------------------------------
PIDy: main=501862  gen=501873  kasjer=501865  wyciag=501868
PIDy: P1=501866  P2=501867  bramki: 501869 501870 501871 501872
```

```
## Fragmenty logów (ciągłość działania)

### kasa.log (widać, że po wznowieniu znowu sprzedaje)
[22:29:44][PID 501865] KASJER: SPRZEDAŻ id_klienta=9322 pid=511206 -> karnet=6565 typ=TK3 (120min) cena=37.50 zł vip=0
[22:29:44][PID 501865] KASJER: sprzedano karnet id_karnetu=6565 typ=TK3 (120min) cena=37.50 zł dla klienta id=9322 (pid=511206)
[22:29:45][PID 501865] KASJER: klient id=10855 pid=512739 wiek=49 typ=ROWER vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=10855 pid=512739 -> karnet=6568 typ=Jednorazowy cena=5.00 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6568 typ=Jednorazowy cena=5.00 zł dla klienta id=10855 (pid=512739)
[22:29:45][PID 501865] KASJER: klient id=7469 pid=509353 wiek=16 typ=ROWER vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=7469 pid=509353 -> karnet=6569 typ=TK2 (60min) cena=35.00 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6569 typ=TK2 (60min) cena=35.00 zł dla klienta id=7469 (pid=509353)
[22:29:45][PID 501865] KASJER: klient id=7932 pid=509816 wiek=44 typ=PIESZY vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=7932 pid=509816 -> karnet=6570 typ=TK3 (120min) cena=50.00 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6570 typ=TK3 (120min) cena=50.00 zł dla klienta id=7932 (pid=509816)
[22:29:45][PID 501865] KASJER: klient id=7094 pid=508978 wiek=46 typ=PIESZY vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=7094 pid=508978 -> karnet=6571 typ=TK3 (120min) cena=50.00 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6571 typ=TK3 (120min) cena=50.00 zł dla klienta id=7094 (pid=508978)
[22:29:45][PID 501865] KASJER: klient id=7004 pid=508888 wiek=6 typ=PIESZY vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: klient id=6575 pid=508459 wiek=79 typ=ROWER vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=6575 pid=508459 -> karnet=6572 typ=Jednorazowy cena=3.75 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6572 typ=Jednorazowy cena=3.75 zł dla klienta id=6575 (pid=508459)
[22:29:45][PID 501865] KASJER: klient id=7042 pid=508926 wiek=91 typ=ROWER vip=0 dzieci=1 (7,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=7042 pid=508926 -> karnet=6573 typ=TK2 (60min) cena=26.25 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6573 typ=TK2 (60min) cena=26.25 zł dla klienta id=7042 (pid=508926)
[22:29:45][PID 501865] KASJER: klient id=10162 pid=512046 wiek=71 typ=ROWER vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=10162 pid=512046 -> karnet=6575 typ=Dzienny cena=75.00 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6575 typ=Dzienny cena=75.00 zł dla klienta id=10162 (pid=512046)
[22:29:45][PID 501865] KASJER: klient id=7370 pid=509254 wiek=18 typ=PIESZY vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=7370 pid=509254 -> karnet=6576 typ=Jednorazowy cena=5.00 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6576 typ=Jednorazowy cena=5.00 zł dla klienta id=7370 (pid=509254)
[22:29:45][PID 501865] KASJER: klient id=9166 pid=511050 wiek=96 typ=ROWER vip=0 dzieci=0 (0,0)
[22:29:45][PID 501865] KASJER: SPRZEDAŻ id_klienta=9166 pid=511050 -> karnet=6577 typ=Dzienny cena=75.00 zł vip=0
[22:29:45][PID 501865] KASJER: sprzedano karnet id_karnetu=6577 typ=Dzienny cena=75.00 zł dla klienta id=9166 (pid=511050)

### main.log (uruchomione procesy, koniec dnia)
[22:29:10][PID 501862] Kasjer uruchomiony (PID=501865)
[22:29:45][PID 501862] Czas symulacji (35 sek) upłynął (elapsed=35)
[22:29:45][PID 501862] === PROCEDURA KOŃCA DNIA ===
[22:29:55][PID 501862] === PROCEDURA KOŃCA DNIA ZAKOŃCZONA ===

## Interpretacja (do prezentacji)
- SIGSTOP kasjera zatrzymuje obsługę MQ kasa → w monitorze widać rosnące msg_qnum (backlog).
- Po SIGCONT proces wraca i backlog maleje, co pokazuje że komunikaty nie znikają i są obsługiwane.
```

Test 7 – Edge case: ubicie klienta na peronie (SEM_UNDO)

Cel: pokazać, że gdy klient zginie na peronie (np. `SIGKILL`), to dzięki `SEM_UNDO` nie zostawia „zajętych na zawsze” slotów i inni klienci nadal mogą wsiadać.

Procedura testu (co robi skrypt): 1. Uruchamia symulację i zatrzymuje wyciąg (`SIGSTOP`) — klienci kumulują się na peronie. 2. Znajduje żywego klienta w stanie `NA_PERONIE` (trzyma sloty SEM_PERON przez `SEM_UNDO`). 3. Wysyła `SIGKILL` do tego klienta — jądro automatycznie oddaje sloty semafora (`SEM_UNDO`). 4. Wznawia wyciąg (`SIGCONT`) i czeka na koniec symulacji.

Uruchomienie (przykład):

```
./tests/test7_sigterm_klient_na_peronie_semundo.sh
```

Sprawdzenia (w `summary.txt`): - po ubiciu klienta system dalej generuje `BOARD/ARRIVE` (dowód ciągłości — `SEM_UNDO` zadziałał), - snapshoty z `./monitor` przed i po kill (jeśli dostępny).

Oczekiwany wynik: po ubiciu jednego klienta w logach nadal pojawiają się `BOARD/ARRIVE`, a koniec dnia przechodzi bez blokady.

test7_sigterm_klient_na_peronie_semundo

Cel: klient ginie NA_PERONIE (po sem_wait_n_undo), ale dzięki SEM_UNDO nie blokuje PERON i system działa dalej.

Wybrany klient: id=81 pid=527649
Wyciąg PID: 527555 (STOP przed kill, CONT po kill)

Snapshot MONITOR (przed kill)
```
KOLEJ KRZESEŁKOWA - MONITOR
------------------------------------------------------------
Faza dnia: OPEN   awaria=0   panic=0
Czas do konca dnia: 23 s   (start=1771882195, koniec=1771882220)
------------------------------------------------------------
Liczniki: teren=40  peron=4  w_krzesle=4  gora=48  aktywni_klienci=174
Statystyki: wygenerowani=175  przejazdy=37  przychod=4972.50 zl
------------------------------------------------------------
Semafory: TEREN=0  PERON=0  BARIERA_AWARIA=0
------------------------------------------------------------
MQ: kasa=0/16384  kasa_odp=0/16384  bramka=113/16384  bramka_odp=0/16384
MQ: peron=0/16384  peron_odp=0/16384  wyciag_req=2/16384  wyciag_odp=0/16384  prac=0/16384
------------------------------------------------------------
PIDy: main=527549  gen=527560  kasjer=527552  wyciag=527555
PIDy: P1=527553  P2=527554  bramki: 527556 527557 527558 527559
```

Snapshot MONITOR (po kill)
```
KOLEJ KRZESEŁKOWA - MONITOR
------------------------------------------------------------
Faza dnia: DRAINING   awaria=0   panic=1
Czas do konca dnia: 23 s   (start=1771882195, koniec=1771882220)
------------------------------------------------------------
Liczniki: teren=38  peron=6  w_krzesle=4  gora=48  aktywni_klienci=134
Statystyki: wygenerowani=175  przejazdy=37  przychod=4972.50 zl
------------------------------------------------------------
Semafory: TEREN=2  PERON=0  BARIERA_AWARIA=0
------------------------------------------------------------
MQ: kasa=0/16384  kasa_odp=0/16384  bramka=73/16384  bramka_odp=0/16384
MQ: peron=0/16384  peron_odp=40/16384  wyciag_req=3/16384  wyciag_odp=0/16384  prac=0/16384
------------------------------------------------------------
PIDy: main=527549  gen=527560  kasjer=527552  wyciag=527555
PIDy: P1=527553  P2=527554  bramki: 527556 527557 527558 527559
```
Asercje
- Ubity klient: BOARD=1, ARRIVE=0
  (BOARD>0: race condition SIGSTOP vs wyciag 1ms - klient zdążył wsiąść, to normalne)
- System: BOARD_total=119, ARRIVE_total=59 (oczekiwane >0 = dowód ciągłości)

klienci.log (fragmenty dla klienta)
```
205:[22:29:55][PID 527649] KLIENT 81: start pid=527649 wiek=100 typ=PIESZY vip=0 dzieci=1 (1,0) rozmiar_grupy=2
206:[22:29:55][PID 527649] DZIECKO 1: razem z opiekunem -> START
209:[22:29:55][PID 527649] KLIENT 81: kupił karnet id_karnetu=48 typ=TK1 (30min) czas_waznosci=1800s vip=0
210:[22:29:55][PID 527649] DZIECKO 1: razem z opiekunem -> BRAMKA1
211:[22:29:55][PID 527649] KLIENT 81: id_karnetu=48 -> BRAMKA1 nr=3 (vip=0, grupa=2)
807:[22:29:55][PID 527649] KLIENT 81: BRAMKA1 OK (nr=3) - jestem na terenie
808:[22:29:55][PID 527649] KLIENT 81: czekam na peron (sloty=2, bramka2=2)
809:[22:29:55][PID 527649] KLIENT 81: prosi PRACOWNIK1 o wejście na peron (bramka2=2 sloty=2)
844:[22:29:56][PID 527649] KLIENT 81: PRACOWNIK1 pozwolił wejść na peron
1033:[22:29:56][PID 527649] KLIENT 81: NA_PERONIE (sloty=2) - czekam na BOARD
```

wyciag.log (końcówka)
```
[22:29:55][PID 527555] WYCIAG: Start (INTERWAL=1ms, PRZEJAZD=9ms, RZEDOW=18, SLOTY/RZAD=4)
[22:29:58][PID 527555] WYCIAG: Drenowanie zakończone (w_krzesle=0, kolejka=0, peron=0, teren=0) - wyłączam za 3s
[22:30:01][PID 527555] WYCIAG: Kończę pracę
```

Interpretacja (do prezentacji)
- 'NA_PERONIE' oznacza, że klient już pobrał sloty SEM_PERON (SEM_UNDO).
- Po SIGKILL sloty wracają automatycznie (SEM_UNDO), a system nadal generuje BOARD/ARRIVE dla innych.

Test 8 – Edge case: crash main i sprzątacz IPC (crash-safe cleanup)

Cel: zademonstrować, że po „twardym” zabiciu `main` (np. `SIGKILL`) zasoby SysV IPC (SHM/SEM/MQ) zostają wyczyszczone, dzięki czemu następne uruchomienie startuje czysto.

Procedura testu (co robi skrypt): 1. Kompiluje helper `ftok`, który oblicza klucze IPC (base + offset) — identyczne jak w `ipc.c`. 2. Uruchamia symulację, sprawdza że obiekty IPC istnieją w `ipcs`. 3. Wysyła `SIGKILL` do `main`. 4. **Faza 1:** czeka do 10s na sprzątacza (PDEATHSIG → cleanup). 5. **Faza 2:** jeśli IPC nadal istnieje a sprzątacz żyje — wysyła mu `SIGUSR1` (force cleanup). 6. **Faza 3:** fallback — dobija osierocone procesy i ręcznie czyści IPC przez `ipcrm`.

Uruchomienie (przykład):

```
./tests/test8_crash_main_sprzatacz_cleanup.sh
```

Sprawdzenia (w `summary.txt`): - obiekty IPC po kluczach z `FTOK_FILE` znikają po crashu, - brak procesów symulacji na liście po zakończeniu testu.

Oczekiwany wynik: IPC wyczyszczone (idealnie przez sprzątacza, w razie potrzeby przez fallback). Test przechodzi jeśli IPC nie zostaje na stałe.

```
# test8_crash_main_sprzatacz_cleanup

Cel: crash-safe cleanup: po SIGKILL main sprzątacz usuwa SysV IPC (SHM/SEM/MQ).

FTOK_FILE: /tmp/kolej_krzeselkowa_ipc.ftok
Sprzątacz PID: 527898

## Klucze (base + offset)
```
SEM 0x4b300b6b
SHM 0x4b300b6c
MQ_KASA 0x4b300b6d
MQ_KASA_ODP 0x4b300b6e
MQ_BRAMKA 0x4b300b6f
MQ_BRAMKA_ODP 0x4b300b70
MQ_PRAC 0x4b300b71
MQ_WYCIAG_REQ 0x4b300b72
MQ_WYCIAG_ODP 0x4b300b73
MQ_PERON 0x4b300b74
MQ_PERON_ODP 0x4b300b75
```

## ipcs PRZED crashem
OK: znaleziono obiekt(y) IPC po kluczach (found_before=1).

## ipcs PO crashu
OK: sprzątacz sam wyczyścił IPC (idealne zachowanie).

## Procesy (best-effort)
OK: brak procesów symulacji na liście.

## Interpretacja (do prezentacji)
- Pokazuje mechanizm crash-safe cleanup: sprzątacz (PDEATHSIG) + fallback (SIGUSR1/ipcrm).
- Sprzątacz samodzielnie wyczyścił IPC po SIGKILL main.
```

Test 9 – Zakres w_krzesle i drenowanie do zera

Cel: potwierdzenie dwóch właściwości: 1. Liczba zajętych krzesełek (`w_krzesle`) nigdy nie spada poniżej 0 ani nie przekracza limitu `(LICZBA_RZEDOW / 2) × KRZESLA_W_RZEDZIE` (= 36 dla 18×4). 2. Po zakończeniu fazy DRAINING `w_krzesle` osiąga dokładnie 0 (wyciąg opróżnił wszystkie krzesełka).

Uruchomienie (przykład):

```
./tests/test9_wkrzesle_range_i_drain_zero.sh
```

Procedura testu (co robi skrypt): 1. Odczytuje `LICZBA_RZEDOW` i `KRZESLA_W_RZEDZIE` z `config.h` i oblicza `MAX_WKRZESLE`. 2. Uruchamia symulację z dużym ruchem. 3. Co 0,2 sekundy pobiera snapshot z `./monitor --once --no-color` i sprawdza wartość `w_krzesle`. 4. Po zakończeniu sprawdza w `wyciag.log` linię `Drenowanie zakończone (w_krzesle=0...)`.

Sprawdzenia (w `summary.txt`): - `w_krzesle` w całym biegu mieściło się w zakresie `[0..MAX_WKRZESLE]`. - Po DRAINING `w_krzesle=0` (dowód pełnego opróżnienia).

Oczekiwany wynik: brak naruszeń zakresu, `w_krzesle=0` po drenowaniu.

```
# test9_wkrzesle_range_i_drain_zero

Cel: (1) w_krzesle nigdy nie spada poniżej 0 ani nie przekracza limitu, (2) po DRAINING jest 0.

Limit z config.h: MAX_WKRZESLE=36 (= (LICZBA_RZEDOW/2)*KRZESLA_W_RZEDZIE)
Zaobserwowane: min=0 max=4

## Wynik
OK: w_krzesle w całym biegu mieściło się w [0..36].

## Dowód drenowania do zera
```
[22:30:30][PID 528243] WYCIAG: Drenowanie zakończone (w_krzesle=0, kolejka=0, peron=0, teren=0) - wyłączam za 3s
```

## Interpretacja (do prezentacji)
- w_krzesle to liczba zajętych miejsc na odcinku w górę, liczona w slotach (waga_slotow).
- DRAINING kończy się dopiero gdy kolejka i ring są puste → w_krzesle=0.
```

Uruchamianie wszystkich testów

Wszystkie 9 testów można uruchomić jednym poleceniem:

```
cd tests && ./run_all.sh
```

Skrypt wyświetla podsumowanie: ile testów przeszło (PASS), ile nie (FAIL), i listę failujących.

6. Linki do istotnych fragmentów kodu (wymagane konstrukcje)

Baza (commit SHA do permanentnych linków):
`b00143740ebfbfe3b1088275fdbe000ade274289`

a) **Tworzenie i obsługa plików** *(creat(), open(), close(), read(), write(), unlink())*

- `open()/close()/read()/write()` – mechanizm "owner lock" + zapis/odczyt flag stanu: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/main.c#L121-L184
- `open(O_CREAT, …)` + `close()` – przygotowanie plików logów: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/main.c#L536-L559
- `creat()` / `unlink()` – **nieużyte jawnie** (tworzenie plików realizowane przez `open(O_CREAT, …)`).

b) **Tworzenie procesów** *(fork(), exec(), exit(), wait())*

- `fork()` + `execv()` + `_exit()` – uruchamianie procesów stałych: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/main.c#L561-L603
- `fork()` + `execv()` + `_exit()` – generator uruchamia proces klienta: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/generator.c#L191-L224
- `waitpid(WNOHANG)` – sprzątanie zombie (main): https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/main.c#L104-L106
- `waitpid(WNOHANG)` – zbieranie zakończonych klientów (generator): https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/generator.c#L37-L53

c) **Tworzenie i obsługa wątków** *(pthread_create(), pthread_join(), …)*

- `pthread_create()` / `pthread_join()` / `pthread_exit()` oraz `pthread_mutex_*` i `pthread_cond_*`
  (wątki "dzieci" zsynchronizowane z opiekunem – bez udziału w IPC): https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/klient.c#L40-L170
- Flagi kompilacji/linkowania `-pthread`: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/Makefile#L6-L7
- `pthread_detach()` / `pthread_mutex_trylock()` / `pthread_cond_signal()` / `pthread_cond_broadcast()` –
  częściowo: `pthread_cond_broadcast()` jest użyte; pozostałe (detach/trylock/signal) nie są potrzebne w tym wariancie.

d) **Obsługa sygnałów** *(kill(), raise(), signal(), sigaction())*

- `sigaction()` – instalacja handlerów w `main`: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/main.c#L323-L361
- `kill(-pgid, …)` – sprzątacz ubija grupę procesów symulacji: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/sprzatacz.c#L193-L199
- `raise()` / `signal()` – **nieużyte** (zamiast tego konsekwentnie `sigaction()`).

e) **Synchronizacja procesów** *(ftok(), semget(), semctl(), semop())*

- `ftok()` + `semget()` + `semctl(SETALL/IPC_RMID)` – tworzenie semaforów (minimalne prawa `IPC_PERMS`): https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/ipc.c#L121-L171 oraz definicja praw: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/config.h#L194
- `semop()` – operacje P/V (mutexy, peron, try-wait): https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/ipc.c#L531-L678

f) **Łącza nazwane i nienazwane** *(mkfifo(), pipe(), dup(), dup2(), popen())*

- `dup2()` – przekierowanie `stdout/stderr` procesu do pliku logów (dla procesów uruchamianych przez main): https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/main.c#L574-L589
- `dup2()` – przekierowanie logów klientów w generatorze: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/generator.c#L56-L73
- `mkfifo()` / `pipe()` / `popen()` – **nieużyte** w tym projekcie.

g) **Segmenty pamięci dzielonej** *(ftok(), shmget(), shmat(), shmdt(), shmctl())*

- `shmget()` + `shmat()` – tworzenie/attach SHM: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/ipc.c#L174-L205
- `shmdt()` + `shmctl(IPC_RMID)` – odłączanie/usuwanie SHM: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/ipc.c#L321-L339

h) **Kolejki komunikatów** *(ftok(), msgget(), msgsnd(), msgrcv(), msgctl())*

- `msgget()` + `msgctl(IPC_RMID)` (na `EEXIST`) – tworzenie kolejek: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/ipc.c#L208-L309
- `msgctl(IPC_RMID)` – cleanup kolejek: https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/ipc.c#L351-L388
- `msgsnd()` / `msgrcv()` – wrappery send/recv (obsługa EINTR/EIDRM, try-nowait): https://github.com/DuckyMein/kolej-linowa/blob/b00143740ebfbfe3b1088275fdbe000ade274289/ipc.c#L684-L730

i) **Gniazda** *(socket(), bind(), listen(), accept(), connect())*

- **NIE DOTYCZY** – projekt korzysta z IPC System V (SHM + SEM + MQ), bez socketów.

7. Repozytorium

Link do repo: https://github.com/DuckyMein/kolej-linowa